

Profile & Settings Backend Requirements

Sep 23, 2026 · @Adeyemi

Everything on the Profile and Account Settings pages beyond core auth (name, phone, state, email, avatar) is client-side mock data with no backend behind it. This memo lists exactly what's real today versus what needs a real endpoint, organized so it can be ticketed directly.

What's real today

The only backend that exists is `/auth/*`, tied to the `AuthUser` record (`id`, `first_name`, `last_name`, `email`, `phone_number`, `nin_masked`, `state`, `role`, `roles`, `is_admin`, `is_active`, `is_email_verified`, `profile_picture`, `theme`, `preferred_language`, `created_at`). Everything below that -- services, portfolio, reviews, bookings, gigs, KYC, sessions, privacy preferences -- has no endpoint at all; it's `localStorage` or plain React state.

Endpoint	Used by frontend?
POST <code>/auth/register</code>	Yes
POST <code>/auth/verify-email</code>	Yes
POST <code>/auth/resend-otp</code>	Yes
POST <code>/auth/login</code> (form, supports <code>totp_code</code>)	Yes
POST <code>/auth/revoke-sessions</code>	Yes (on logout)
POST <code>/auth/forgot-password</code>	Yes
POST <code>/auth/reset-password</code>	Yes
GET <code>/auth/me</code>	Yes
PUT <code>/auth/me</code>	Yes
DELETE <code>/auth/me</code>	Yes (Delete Account)
POST <code>/auth/me/picture</code>	Yes

Endpoint	Used by frontend?
POST /auth/deactivate-me	No -- client method exists, nothing calls it
POST /auth/2fa/setup	No -- client method exists, nothing calls it
POST /auth/2fa/verify	No -- client method exists, nothing calls it

The last three rows matter for scoping: before asking backend for deactivation or 2FA, confirm these already work end-to-end -- if so this is a frontend wiring task, not a backend one.

Client Profile (ProfileView.tsx)

Field	Status	Notes
First name, last name, phone, state	Real	PUT /auth/me , works today
Profile picture	Real	POST /auth/me/picture , works today
Email, email-verified badge, member-since date	Real, read-only	Displayed from GET /auth/me ; there is no email-change flow anywhere (see below)
Jobs Completed / Active Bookings counts	Mock	Computed client-side from a local mock bookings array. Needs: a bookings/jobs API scoped to the authenticated client, or at minimum a count endpoint
Saved Artisans (list + count)	Mock	Pure localStorage , no favorites concept server-side. Needs: a saved/favorites endpoint (add, remove, list) keyed to the client

Known gap, not a backend ask: the email field renders a draft input as if editable, but the save handler never includes email in its PUT /auth/me payload -- edits silently do nothing. Decide product-side whether email should be changeable at all; if yes, backend

needs to say whether that's a plain field update or requires re-verification (most APIs treat email change as security-sensitive and gate it behind a new OTP).

Artisan Profile (`ProProfileManagement.tsx`)

Field	Status	Notes
First name, last name, phone	Real	<code>PUT /auth/me</code> , works today
Profile picture	Real	<code>POST /auth/me/picture</code> , works today
Base location / neighborhood	Mock, deliberately not wired	It's a free-text "Neighborhood, State" display string with no reliable way to split a clean <code>state</code> value back out of it. Needs: a real structured location field (state dropdown + separate neighborhood text) before this can round-trip safely
Email	Real, read-only	Same as client side -- no change flow
Tagline, bio, trade category	Mock	No backend field exists at all. Needs: these added to the artisan's profile record
Services & pricing (list of offerings, price, duration)	Mock	Hardcoded seed data, CRUD is local state only. Needs: a services resource (create/edit/delete) scoped to the artisan
Work portfolio (title, image, description, category, date)	Mock	"Image upload" is actually just a free-text URL field, not a real upload. Needs: a portfolio resource + real image upload (not just URL paste)

Field	Status	Notes
KYC / identity verification, "Verified Pro" badge	Mock	Entirely <code>localStorage</code> (<code>kazihub_kyc_completed_<id></code>) -- there is no document upload, no liveness check, no verification workflow of any kind server-side. Needs: a real KYC pipeline (document upload, status, admin review or a verification vendor integration)
Jobs completed, average rating, review count, years of experience	Mock	Read straight off seed data. Needs: these derived from real bookings/reviews once those exist, not manually editable
"Response time" stat	Hardcoded string	Not even a data field -- literally typed into the JSX ("~15 min reply"). Needs a real definition (e.g. computed from message timestamps) before it's worth a backend field

Account Settings (`SettingsView.tsx` , shared by both roles)

Section	Status	Notes
Change Password	Fully fake	Form validates client-side (fields filled, new matches confirm) then just shows a success toast. No API call of any kind. Needs: an authenticated change-password endpoint (current + new password) -- nothing in the client today calls one
Two-Factor Authentication	Disabled, "Coming soon"	<code>setupTwoFactor</code> / <code>verifyTwoFactorSetup</code> already exist in the API client (<code>/auth/2fa/setup</code> , <code>/auth/2fa/verify</code>) and <code>login</code> already accepts <code>totp_code</code> . This looks backend-ready -- confirm those two endpoints actually work, then it's a frontend task to build the enrollment UI

Section	Status	Notes
Biometric Login	Disabled, "Coming soon"	WebAuthn/passkey feature; needs backend credential storage + challenge/verify endpoints if pursued. Not urgent
Active Devices & Sessions	Mock	Shows only "this device," with copy admitting multi-device tracking isn't built. Needs: a sessions-list endpoint (device/IP/last-active) + a per-session revoke, if this is wanted
Telephone number privacy (who can see my phone)	Not persisted at all	Not even <code>localStorage</code> -- plain React state, resets on every page reload. Needs: a field on the account (or a settings sub-resource) to persist this, since it's meant to gate real contact-info visibility
Neighborhood sharing toggle	Not persisted at all	Same as above
Data export ("Download My Data")	Partially real	Triggers a real browser download, but the payload is just name/email/phone/state/booking-count assembled client-side -- not an actual data export. Needs: a real export endpoint once there's real data to export
Freeze account	Fully fake	Pure local <code>isFrozen</code> toggle + toast. No backend call -- the account is never actually paused. Needs: wire to <code>POST /auth/deactivate-me</code> , which already exists but is unused (see "What's real today") -- confirm it does what "freeze" implies (reversible, hides from search) before wiring it
Delete account	Real	<code>DELETE /auth/me</code> , works today
Terms of Service / Privacy Policy	Static copy	Hardcoded in the frontend, not backend-driven. Only worth flagging if these need to be versioned/CMS-managed later

Also found: `sessionTimeout`, `publicReviews`, and `marketingAnalytics` are declared as state in this file but never rendered anywhere -- dead code, not a backend gap. Worth a quick frontend cleanup pass, or confirms these were planned settings that got cut.

Consolidated ask, by priority

P0 -- confirm before building anything (may already exist)

1. Does `POST /auth/2fa/setup` / `POST /auth/2fa/verify` actually work end-to-end? If yes, this is a frontend-only task.
2. Does `POST /auth/deactivate-me` do what "freeze account" needs (reversible, hides the artisan from search, preserves history)? If yes, this is a frontend-only task.

P1 -- new endpoints needed, core account security

1. Change password while authenticated (current password + new password).
2. Sessions list + per-session revoke, for "Active Devices & Sessions."
3. Persistence for privacy preferences (phone visibility rule, neighborhood-sharing toggle) -- a settings sub-resource on the account, or fields on `AuthUser` / `UserUpdate`.

P2 -- new resources needed, artisan-facing (largest scope)

1. Services & pricing (CRUD, scoped to artisan).
2. Portfolio items (CRUD + real image upload, not a URL-paste field).
3. KYC/identity verification pipeline (document upload, status, review workflow).
4. Structured location on the artisan profile (state + neighborhood as separate fields, not one free-text string) -- blocks safely saving location edits at all.
5. Tagline, bio, and trade category as real profile fields.

P3 -- depends on other systems that don't exist yet either

1. Bookings/jobs API (blocks: client's Jobs Completed/Active Bookings counts, artisan's completed-jobs/rating/review stats -- these should probably derive from booking and review data once that exists, not be independently editable).
2. Saved/favorites endpoint (client's Saved Artisans).
3. Real data export endpoint (once there's real data worth exporting).

Product decision needed, not purely backend

Should users be able to change their email at all? The UI currently implies yes but silently no-ops. If yes, this is normally a security-sensitive flow (re-verification via OTP), not a plain field update -- needs a decision before an endpoint is designed.